

Ejemplo 1 (XMLHttpRequest): Obtener usuarios de una API pública

Objetivo del ejercicio

- Aprender a realizar **peticiones AJAX asíncronas** con XMLHttpRequest.
- Mostrar cómo funcionan los **estados de la petición**, readyState y status.
- Mostrar los datos obtenidos en la pantalla de forma dinámica usando Bootstrap.

Código completo (HTML + Bootstrap + JS)

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo 1 - AJAX con XMLHttpRequest</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">

  <h1 class="mb-4">Ejemplo 1: Obtener Usuarios con
XMLHttpRequest</h1>
  <p>Haz click en el botón para cargar usuarios desde la API pública
<code>JSONPlaceholder</code> usando <strong>AJAX</strong>.</p>

  <button id="btnCargarUsuarios" class="btn btn-primary">Cargar
Usuarios</button>

  <div id="resultadoUsuarios" class="mt-3"></div>

  <script>
    const btnCargarUsuarios =
document.getElementById('btnCargarUsuarios');
    const resultadoUsuarios =
document.getElementById('resultadoUsuarios');

    btnCargarUsuarios.addEventListener('click', () => {
      // Limpiar resultados anteriores y mostrar mensaje de
carga
      resultadoUsuarios.innerHTML = `<div class="alert alert-
info">Cargando usuarios...</div>`;

      // 1. Crear objeto XMLHttpRequest
      const xhr = new XMLHttpRequest();

      // 2. Configurar la petición: método GET, URL y asincronía
      xhr.open('GET',
'https://jsonplaceholder.typicode.com/users', true);

      // 3. Definir función que se ejecuta cuando cambia el
estado de la petición
      xhr.onreadystatechange = () => {
```

```

        // Comprobar que la petición se completó
        if (xhr.readyState === 4) { // 4 = petición completada
            if (xhr.status === 200) { // 200 = OK
                // Convertir la respuesta JSON a objeto JS
                const usuarios = JSON.parse(xhr.responseText);

                // Construir la lista de usuarios
                let html = '<ul class="list-group">';
                usuarios.forEach(user => {
                    html += `<li class="list-group-item"><strong>${user.name}</strong> - ${user.email}</li>`;
                });
                html += '</ul>';

                resultadoUsuarios.innerHTML = html;
            } else {
                // Mostrar mensaje de error si la petición
                // falla
                resultadoUsuarios.innerHTML = `<div
                class="alert alert-danger">Error: ${xhr.status} -
                ${xhr.statusText}</div>`;
            }
        }
    };

    // 4. Enviar la petición al servidor
    xhr.send();
});
</script>

</body>
</html>

```

Explicación paso a paso

1. **Crear objeto XMLHttpRequest:**
2. `const xhr = new XMLHttpRequest();`
 - o Este objeto permite **realizar peticiones HTTP** de forma asíncrona.
3. **Configurar la petición:**
4. `xhr.open('GET', 'https://jsonplaceholder.typicode.com/users', true);`
 - o 'GET' es el método HTTP.
 - o La URL es la API pública.
 - o `true` indica que la petición será **asíncrona**.
5. **Manejo de cambios de estado:**
6. `xhr.onreadystatechange = () => { ... }`
 - o `xhr.readyState` indica el **estado de la petición (0-4)**.
 - o `xhr.status` indica el **código HTTP de la respuesta**.
 - o Solo cuando `readyState === 4` y `status === 200` sabemos que la respuesta es correcta.
7. **Procesar la respuesta:**
8. `const usuarios = JSON.parse(xhr.responseText);`
 - o La respuesta de la API llega como **texto**, se convierte a **objeto JS** con `JSON.parse()`.
9. **Mostrar resultados en el DOM:**

- Se construye un `<ul>` con los nombres y correos de los usuarios usando Bootstrap para estilo.

10. Enviar la petición:

11. `xhr.send()`;

- La petición se envía al servidor.

Preguntas de comprensión para alumnos

1. ¿Qué diferencia hay entre `XMLHttpRequest` y `fetch`?
2. ¿Qué significa `xhr.readyState` y cuáles son sus valores posibles?
3. ¿Qué pasa si la API devuelve un error 404 o 500?
4. ¿Por qué necesitamos convertir la respuesta con `JSON.parse()`?
5. ¿Cómo podrías mostrar más información de los usuarios, como la ciudad o la empresa?

Mejoras sugeridas para el alumno

1. Mostrar un **spinner de carga** mientras se realiza la petición.
2. Añadir un **campo de búsqueda** para filtrar usuarios por nombre.
3. Mostrar los datos en **tarjetas de Bootstrap** en lugar de lista.
4. Manejar otros **estados de readyState** para mostrar información en tiempo real.
5. Implementar la misma petición pero usando **promesas** para comparar la sintaxis moderna.